

PERSON 1 · DATA, MODEL AND INFERENCE ASSURANCE

Handover brief for Person 2

What already exists, which functions to call, and which shift and comparator work is still yours. Source of truth is the code under Nethra-guard/backend.

This document is an integration brief. It does not add features. Read it before writing distribution-shift or comparator code. Do not reimplement dataset parsing, model hashing, or the ledger.

1. Ownership

Person 1 produces validated dataset and model findings, asset fingerprints, a hash chain, and a normalized detection list. Person 2 consumes those outputs to measure distribution shift, compare two runs, and produce an explainable overall assurance status.

- **Person 1, done:** dataset integrity, model fingerprinting, simulated inference output, hash-chain create and verify.
- **Person 2, still to build:** image-feature shift, baseline-versus-current comparison, and the overall status across modules.
- In every Person 1 scenario, `distribution_shift` and `comparator` are returned as `NOT_RUN` with a reason string. That is a skipped module, not a failed shift test. Do not show it as `PASS`, and do not hide it.
- Reason strings, copied from the runner: “Distribution shift is owned by the analytics workstream and is not executed in this module.” and “Baseline comparison is owned by the analytics workstream and is not executed in this module.”

2. How to run

From the repository root the backend lives in `Nethra-guard/backend`. The checks are offline. Pytest does not need a network once the packages below are installed.

```
cd Nethra-guard/backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pytest
python -m assurance.scenarios NONE
python -m assurance.scenarios DATA_ANOMALY
python -m assurance.scenarios MODEL_MISMATCH
```

A good pytest run reports 23 passed. Runtime dependencies are pydantic, Pillow, and pytest. Call these functions. Do not scrape the React mock data.

```
from assurance.data_integrity import check_dataset, fingerprint_dataset
from assurance.model_integrity import check_model
from assurance.inference import run_inference, normalized_detections
from assurance.scenarios import run_scenario, scenario_to_handoff
from assurance.ledger import append_entry, verify_ledger
from assurance.hashing import sha256_file, sha256_text, canonical_json

check_dataset(dataset_dir: Path) -> ModuleResult
fingerprint_dataset(dataset_dir: Path) -> str
check_model(model_path, *, baseline_sha256, allowed_root) -> ModuleResult
run_inference(model_path, image_paths, *, allowed_root, config=None) -> InferenceResult
normalized_detections(result) -> list[dict]
run_scenario(scenario, ledger=None, timestamp=None) -> ScenarioRun
scenario_to_handoff(run) -> dict
append_entry(ledger, *, run_id, timestamp, dataset_hash, model_hash,
              config_hash, summary_hash, actor_label='local-demo',
```

```

        baseline_run_id=None) -> LedgerEntry
verify_ledger(entries) -> LedgerVerification
canonical_json(payload) -> str    # sorted keys, no extra whitespace, UTF-8

```

3. File map

All paths below are relative to `Nethra-guard/backend`. Images are synthetic flat-color PNGs, 100 by 80 pixels. They are not photographs from the public COCO dataset.

- `fixtures/clean/` — four PNGs (`img_001.png` through `img_004.png`) plus `annotations.json`. This is the known-good set.
- `fixtures/anomaly/` — controlled bad set. `img_001.png` and `img_001_dup.png` are the same bytes. `img_003.png` is listed in the JSON and the file is absent. One box extends outside the image. One annotation uses category id 99.
- `fixtures/models/baseline_model.bin` and `altered_model.bin` — short stand-in files, not neural-network weights.
- `fixtures/baseline_manifest.json` — trusted hashes for the clean dataset and the baseline model file.
- `fixtures/expected_clean.json` and `fixtures/expected_anomaly.json` — full handoff documents for those two scenarios, including modules, inference, and one ledger entry.
- `fixtures/normalized_detections.json` — the detection list for the clean image set. Start here.

Live manifest values:

```

annotation_format: coco
clean_dataset_hash:
  c854628a8703dc247e167b82714cc93e6fc4b30fa948aabc7e2867b16c88dfa5
baseline_model_hash:
  6dc1d983695b77a964a28d6b732632473e4c7f167d9f055f7f3f373fc487daff
model_file: models/baseline_model.bin
altered_model_file: models/altered_model.bin

```

The dataset hash is SHA-256 of canonical JSON containing the annotation-file hash and each image-file hash, sorted by file name. It is not a hash of the folder path. Recompute it with `fingerprint_dataset` if you regenerate the PNGs.

4. COCO labels versus detection boxes

These two box formats are different. Do not treat them as the same array.

- Annotation boxes in `annotations.json` are COCO `[x, y, width, height]`. Example on `img_001`: `[10, 10, 30, 20]`.
- Inference boxes are `bbox_xyxy`: `[x1, y1, x2, y2]`. Example for `img_001.png`: `[10, 10, 40, 30]`.
- Classes in this fixture: 1 = vehicle, 2 = person. Category id 99 is invalid and is a Person 1 finding, not a third class.
- Only COCO JSON is parsed. YOLO label files are not supported. Do not add a second parser in the shift code.

5. Detection contract and the simulated warning

Person 2 should join records on `image_id`. The clean contract file is `fixtures/normalized_detections.json`.

```

{
  "image_id": "img_001.png",
  "detections": [
    { "class_id": 1, "confidence": 0.91, "bbox_xyxy": [10, 10, 40, 30] }
  ]
}

```

Clean-set boxes, fixed by the deterministic adapter:

```
img_001.png  class 1  0.91  [10, 10, 40, 30]
img_002.png  class 2  0.83  [12, 8, 32, 48]
img_003.png  class 1  0.77  [15, 10, 40, 35]
img_004.png  class 2  0.88  [8, 10, 26, 40]
```

- Every inference result from the .bin stand-in sets `simulated: true` and `executed: true`. The limitation text says these boxes are not outputs of the model weights. Keep that flag visible in any UI or report.
- Shift metrics may use the PNG pixels (color, brightness, size). Those pixels are real measurements of the fixture images.
- Comparator deltas on these boxes only prove that the contract is stable. They do not prove prediction drift, because the weights were never run. The same image id always returns the same box.
- On the anomaly set, `img_003.png` is missing. It is listed in `skipped_images` and is omitted from the detection list. Do not invent a box for it.
- Calling `run_inference` twice on the same images and config yields the same detections, hashes, and config hash. Ignore `timing_ms` when comparing runs; it is wall-clock time.
- ONNX and YOLO files are refused (`executed: false`). Pickle files raise `UnsafeModelError` and are never deserialized.

6. Module result schema

Module JSON is `snake_case`. This is the contract in `assurance/schemas.py`. The React mock uses different names (`observed`, `testMethod`, category `Data Integrity`). Map fields in the backend. Do not change Person 1's schema to match the mock, and do not invent a 0–100 trust score.

```
{
  "module": "data_integrity | model_integrity | distribution_shift | comparator",
  "status": "PASS | WARNING | FAIL | NOT_RUN",
  "metrics": {},
  "findings": [{
    "severity": "LOW | MEDIUM | HIGH | CRITICAL",
    "title": "Exact duplicate images",
    "method": "Rule-based / deterministic check. ...",
    "observed_value": "2",
    "threshold": "1",
    "evidence_refs": [{
      "id": "exact-duplicate-img_001.png",
      "sample_id": "img_001.png",
      "file_hash": "<sha256>",
      "evidence_type": "exact_duplicate"
    }],
    "limitations": "Exact byte duplicates do not establish malicious poisoning."
  }]
}
```

Status rule inside Person 1 modules: any CRITICAL or HIGH finding makes the module FAIL. Any other finding makes it WARNING. No findings makes it PASS. A skipped module is NOT_RUN with `metrics.reason` and an empty findings list. Methods for Person 1 checks begin with “Rule-based / deterministic check”.

7. Scenario results you can rely on

Run them through `run_scenario(name)`. The frozen handoff timestamp used in the committed JSON files is `2026-01-01T00:00:00Z`.

- **NONE.** Clean dataset and baseline model. `data_integrity` PASS, `model_integrity` PASS, shift NOT_RUN, comparator NOT_RUN. Ledger status VALID. The entry's `datasetHash` and `modelHash` equal the manifest hashes above. `previousHash` is 64 zero characters.
- **DATA_ANOMALY.** Anomaly dataset and baseline model. Data module FAIL. Finding titles, exactly: “Bounding box outside image bounds”, “Exact duplicate images”, “Invalid class id”, “Missing image file”. Model module still PASS, because the baseline file was not changed.

- **MODEL_MISMATCH.** Clean dataset and altered_model.bin. Data PASS. Model FAIL. The single finding title is “SHA-256 hash differs from registered baseline”. observed_value is the altered file's SHA-256. threshold is the baseline model hash. Limitations, verbatim: “A hash mismatch means the artifact differs from the registered baseline. It does not prove malicious tampering.”
- **DISTRIBUTION_SHIFT.** Person 1 does not execute this scenario. Every module is NOT_RUN, inference is null, and no ledger entry is appended. Build the real shift scenario on your side.

8. Thresholds Person 1 already uses

These are prototype demonstration thresholds, not deployment limits. Say that in your own shift thresholds as well.

- Exact duplicate: more than 1 file with the same SHA-256 is WARNING. It does not by itself FAIL the dataset.
- Extreme aspect ratio: max(width/height, height/width) greater than 8 is WARNING.
- Class imbalance: max(class count) / min(class count) at or above 4, and at least two classes present, is WARNING.
- Resolution outlier: width or height more than 50 percent from the median, when at least four images decode, is WARNING.
- FAIL (critical structure): malformed COCO JSON, missing or undecodable image, non-positive box, box outside the image, invalid class id, annotation path that leaves the images directory.
- A missing baseline hash on a model file is NOT_RUN, never PASS. An unreadable model file or a path outside the models directory is FAIL.

9. Ledger

Person 1 creates and verifies entries. A future API layer only stores them. Do not rewrite the hash formula.

```
current_hash = SHA256( canonical_json(entry_without_current_hash) + previous_hash )
genesis_previous_hash = 64 zero characters
canonical_json = json.dumps(obj, sort_keys=True, separators=(',', ':'), ensure_ascii=False)
```

The hashed payload uses these camelCase keys and does not include currentHash or isValid: actorLabel, baselineRunId, configHash, datasetHash, modelHash, previousHash, runId, sequence, summaryHash, timestamp. actorLabel for the demo is local-demo.

When you add a shift or comparator config, hash that config with canonical_json and pass the digest as config_hash into append_entry. Verify with verify_ledger. It returns VALID, or FAILED plus firstInvalidSequence and a reason. Demonstrate tampering on a copied list of entries. Do not edit the live fixture files.

10. What Person 2 still builds

- Reference batch versus current batch, using the same features on both. Suggested features: per-channel RGB mean and standard deviation, brightness, contrast, image width and height, and optionally edge density.
- One documented comparison metric. Population Stability Index on binned features, or a normalized histogram distance. Publish the formula. Label the cutoff as a prototype threshold.
- Controlled fixtures you own: brightness, blur, contrast, color cast, and resolution change. Person 1 did not generate those images. A shift is a warning signal. It does not by itself prove the model failed.
- Comparator: compare two runs only when dataset, model, preprocessing, and configuration are compatible. Otherwise return a not-comparable result. Do not print misleading deltas.
- On compatible runs, report detection-count change, confidence change, class change, and bounding-box IoU for matched detections. Use the xxyy boxes from the detection contract.
- Do not report precision, recall, or mAP unless you have ground truth and a real evaluator. The COCO boxes in the fixture can serve as ground truth only if you say so and you actually compute the metric.

- Overall status, your rule: a critical finding fails the run; warnings mean review; all required checks passing means pass; a required check that did not execute stays NOT_RUN. Do not collapse this into one unexplained score.
- Do not implement, and do not claim, the frontend mock's SSIM near-duplicate method or its KS-test on embeddings. Those strings are placeholder UI copy. Person 1's duplicate check is exact SHA-256 only.

11. Integration checklist

- Read fixtures/normalized_detections.json before writing shift or comparator code.
- Join on image_id. Keep simulated: true visible wherever those boxes are shown.
- Attach Person 1 evidence: sample_id, file_hash, dataset hash, and model hash. Do not drop the limitations string.
- Leave Person 1 findings intact. Add your own findings with module distribution_shift or comparator.
- Hash your config with canonical_json before you append a ledger entry.
- Do not parse YOLO. Do not load pickle or .pt weights. Stay offline. Do not download the public COCO dataset for this demo.
- Do not treat NOT_RUN as PASS. Do not treat a hash mismatch as proof of tampering.

Package version in assurance/__init__.py is 0.1.0. If a fixture file and this brief disagree, trust the file and the pytest suite.